

My Project

v3.0

Generated by Doxygen 1.13.2

1 Trecia versija	1
1.1 Kompiuterio charakteristikos	1
1.2 Kompiliavimo instrukcijos	1
1.3 Programos naudojimosi instrukcijos	1
1.3.0.1 Pagrindinio meniu pasirinkimai:	1
1.3.0.2 Pasirinkus 4 punkta meniu:	2
1.3.0.3 Pastabos:	2
1.4 v.pradine versija	2
1.5 v0.1 versija	2
1.6 v0.2 versija	3
1.7 v0.3 versija	3
1.8 v0.4 versija	3
1.9 v1.0 pradine versija	3
1.10 v1.0 versija	3
1.11 v1.1 versija	3
1.12 v1.2 versija	4
1.12.1 std::vector ir mano vektoriaus klases vidutinio programos veikimo laiko palyginimas	5
1.12.1.1 Pastaba: Kiekvienas greitis - vidutinis 3 testu laikas (sek.).	5
1.13 Rule of Five realizacija	5
1.14 IO operatoriai	5
1.15 Rule of Five ir IO operatorius testai	5
1.16 V1.1 versijos testavimai ir palyginimas su V1.0 versija (Nauja)	5
1.16.0.1 Pastaba: Kiekvienas greitis - vidutinis 3 testu laikas (sek.). Programoje naudojamas konteineris - vektorius. Paleidžiama 3 strategija.	5
1.17 V1.0 versijos testavimai (Sena)	6
1.18 Programos testavimo rezultatai pradiniam v1.0 release	6
1.18.1 1000 Studentų analize	6
1.18.1.1 Nuskaitymas (sek.):	6
1.18.1.2 Rusiavimas faile (sek.):	6
1.18.1.3 Rusiavimas i kietekus ir vargsiukus (sek.):	6
1.18.1.4 Vidurkiai (sek.):	6
1.18.2 10000 Studentu analize	7
1.18.2.1 Nuskaitymas (sek.):	7
1.18.2.2 Rusiavimas faile (sek.):	7
1.18.2.3 Rusiavimas i kietekus ir vargsiukus (sek.):	7
1.18.2.4 Vidurkiai (sek.):	7
1.18.3 100000 Studentu analize	7
1.18.3.1 Nuskaitymas (sek.):	7
1.18.3.2 Rusiavimas faile (sek.):	8
1.18.3.3 Rusiavimas i kietekus ir vargsiukus (sek.):	8
1.18.3.4 Vidurkiai (sek.):	8
1.18.4 1000000 Studentu analize	8

1.18.4.1 Nuskaitymas (sek.):	8
1.18.4.2 Rusiavimas faile (sek.):	8
1.18.4.3 Rusiavimas i kietekus ir vargsiukus (sek.):	9
1.18.4.4 Vidurkiai (sek.):	9
1.18.5 10000000 Studentu analize	9
1.18.5.1 Nuskaitymas (sek.):	9
1.18.5.2 Rusiavimas faile (sek.):	9
1.18.5.3 Rusiavimas i kietekus ir vargsiukus (sek.):	9
1.18.5.4 Vidurkiai (sek.):	10
1.19 Programos rusiavimo i kietekus ir vargsiukus greitis priklausomai nuo strategiju rezultatai	10
1.19.1 1 strategija	10
1.19.1.1 1000 Studentu analize	10
1.19.1.2 Nuotraukos	10
1.19.1.3 10000 Studentu analize	10
1.19.1.4 Nuotraukos	11
1.19.1.5 100000 Studentu analize	11
1.19.1.6 Nuotraukos	11
1.19.1.7 1000000 Studentu analize	11
1.19.1.8 Nuotraukos	12
1.19.1.9 10000000 Studentu analize	12
1.19.1.10 Nuotraukos	13
1.19.2 2 strategija	13
1.19.2.1 1000 Studentu analize	13
1.19.2.2 Nuotraukos	13
1.19.2.3 10000 Studentu analize	13
1.19.2.4 Nuotraukos	14
1.19.2.5 100000 Studentu analize	14
1.19.2.6 Nuotraukos	14
1.19.2.7 1000000 Studentu analize	14
1.19.2.8 Nuotraukos	14
1.19.2.9 10000000 Studentu analize	14
1.19.2.10 Nuotraukos	15
1.19.3 3 strategija	15
1.19.3.1 1000 Studentu analize	15
1.19.3.2 Nuotraukos	15
1.19.3.3 10000 Studentu analize	15
1.19.3.4 Nuotraukos	16
1.19.3.5 100000 Studentu analize	16
1.19.3.6 Nuotraukos	16
1.19.3.7 1000000 Studentu analize	16
1.19.3.8 Nuotraukos	16
1.19.3.9 10000000 Studentu analize	16

1.19.3.10 Nuotraukos	17
1.20 5 failų kūrimas	17
2 Hierarchical Index	19
2.1 Class Hierarchy	19
3 Class Index	21
3.1 Class List	21
4 File Index	23
4.1 File List	23
5 Class Documentation	25
5.1 Studentas Class Reference	25
5.1.1 Member Function Documentation	26
5.1.1.1 getPavarde()	26
5.1.1.2 getVardas()	26
5.1.1.3 setPavarde()	26
5.1.1.4 setVardas()	26
5.2 vector< T > Class Template Reference	27
5.3 Zmogus Class Reference	28
6 File Documentation	29
6.1 santrauka.h	29
6.2 funkcijos.h	29
6.3 studentas.h	30
6.4 vector.h	32
6.5 zmogus.h	35
Index	37

Chapter 1

Trecia versija

1.1 Kompiuterio charakteristikos

Komponentas	Specifikacija
CPU	AMD Ryzen 5 4600H
RAM	8GB DDR4
DISK	SSD (NVMe)

1.2 Kompiliavimo instrukcijos

1. Atsisiusti g++
2. Atsisiusti Make
3. Kopijuoti projekto direktorija `git clone https://github.com/NikaBukolovaite/Objektinis_Programavimas.git` (arba Download zip)
4. Atsidaryti komandine eilute projekto direktorijoje ir irasyti: `make -C Vector`. Jei norima paleisti testus: `make -C Vector testai`.
5. Paleisti norima programos versija: `./Vector/vektoriai.exe`

1.3 Programos naudojimosi instrukcijos

Si programa skirta studentu duomenų valdymui ir analizei, suteikiant ivairias galimybes ivesti, generuoti, rusiuoti ir analizuoti studentu informacija. Programos meniu siulo siuos pasirinkimus:

1.3.0.1 Pagrindinio meniu pasirinkimai:

1. Studentus ir pazymius ivesti ranka Leidzia vartotojui rankiniu budu ivesti studentu vardus ir ju pazymius.
2. Generuoti pazymius, bet studentus ivesti ranka Vartotojas iveda studentu vardus ir kiek viso studentas turi pazymiu, o programa automatiškai sugeneruoja atsitiktinius pazymius tiems studentams.
3. Generuoti pazymius ir studentus Programa automatiškai sugeneruoja studentus su atsitiktiniais vardais, pavardem ir pazymiais.
4. Nuskaitymas is failo Leidzia nuskaityti studentu duomenis is pasirinkto failo, esancio "NuskaitymoFailai" aplankale.
5. Baigti programa Uzbaigius programa bus pateikta programos veikimo suvestine.

1.3.0.2 Pasirinkus 4 punkta meniu:

1. Generuoti faila Vartotojas turi galimybę sugeneruoti pasirinkta failu kieki su vartotojo pasirinktu studentu kiekiu. Pazymius ir ju kieki programa automatiškai generuoja atsitiktinai.
2. Pasirinkti ivesties faila Leidžia vartotojui pasirinkti konkrečia ivesties faila is "NuskaitymoFailai" aplankalo.
3. Testavimas Atlieka programos nasumo testus, iskaitant failu kurimo, nuskaitymo, rusiavimo, kategorizavimo ir isvesties greicio matavimus.

1.3.0.3 Pastabos:

Studentus isvesti galima tokiais budais:

1. I viena bendra faila "rezultatai.txt"
2. I du atskirus failus pagal galutini pazymi - "kietekai.txt" ir "vargsiukai.txt" - vartotojas turi pasirinkima tarp 3 strategiju, kurios nulemia, koku budu bus isrusuoti studentai i du failus.

Noredami naudoti 4 punkta su savo failais, ikelkite juos i "NuskaitymoFailai" aplankala. Testavimo rezime galima ivertinti programos efektyvuma atliekant ivairias operacijas su skirtingo dydzio duomenu rinkiniais

1.4 v.pradine versija

Pradineje versijoje suurta bazine studentu valdymo programa, kuri leido raniniu budu surasyti studentu vardus, pavardes, pazymius ir egzamino rezultata. Turejo funkcijas galutinio pazymio skaiciavimui pagal vidurki arba medianą.

1.5 v0.1 versija

Optimizuota pradine versija, programai nebereikia is anksto zinoti, kiek viso bus studentu ir namu darbu. Taip pat pridetas meniu su 4 pasirinkimais:

1. Ivesti studentu varda, pavarde, pazymius raniniu budu.
2. Sugeneruoti pazymius atsitiktinai ir rankiniu budu surasyti studentu vardus ir pavardes.
3. Atsititinai sugeneruoti visus studentu duomenis ir pazymius.
4. Baigti programa.

Si versija realizuota dviem budais:

1. naudojamas masyvas pazymiu saugojimui;
2. naudojamas std::vector tipo konteineris studentu ir pazymiu saugojimui.

1.6 v0.2 versija

I meniu pridetas 5 pasirinkimas - skaitymas is failo. Taip pat programa testuoja, kaip greitai yra nuskaitomas failas priklausomai nuo jame esancio studentu kiekio. Taip pat atsirado pasirinkimas, ar studentus isvesti i konsolę, ar i faila "rezultatati.txt".

1.7 v0.3 versija

Prideta klaidu gaudymo sistema.

1.8 v0.4 versija

Prideta galimybė generuoti failus is kuriu veliau bus nuskaitoma informacija. Visi failai yra archgyvuojami atsirame aplankale "NuskaitymoFailai". Atsirado pasirinkimas ar visus studentus isvesti i konsolę/faila "rezultatai.txt", ar iskirstyti studentus i 2 atskirus failus "kietekai.txt" ir "vargsiukai.txt" pagal pazymius (galutinis balas < 5.0 ir galutinis balas ≥ 5.0). Taip pat vykdoma programos veikimo greicio (spartos) analizę.

1.9 v1.0 pradine versija

Sukurtos 3 programos su skirtingais konteineriais - `std::vector`, `std::list` ir `std::deque`.

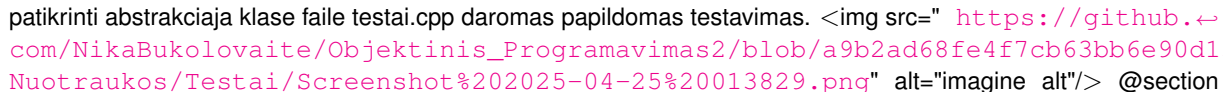
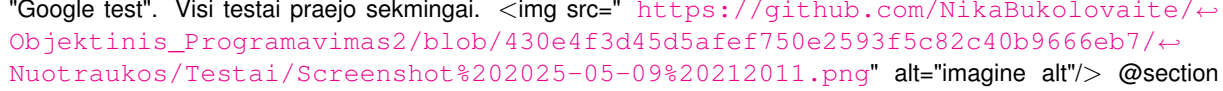
1.10 v1.0 versija

Sukurtos 3 skirtingos strategijos studentu rusiavimui i 2 failus - "kietekai.txt" ir "vargsiukai.txt"

1.11 v1.1 versija

Palikta viena - greiciausia strategija, vieno tipo konteineris (vector), pagreitintas failu nuskaitymas ir kodas perrasytas objektais (naudojant klases).

1.12 v1.2 versija

Igyvendintas „Rule of Five“ ir ivesties/isvesties operatoriai. Padaryti siu metodu testai. @section autotoc_md16 v1.5 versija Sukurta abstrakti klase "Zmogus" ir dabar klase "Studentas" yra isvestine klase is "Zmogus". Kad patikrinti abstrakciaja klase faile testai.cpp daromas papildomas testavimas.  @section autotoc_md17 v2.0 versija Padaryta dokumentacija su Doxygen (html ir latex.pdf) ir realizuoti Unit testai naudojant "Google test". Visi testai praejo sekmingai.  @section autotoc_md18 v3.0 versija Sukurta nuosava vektoriaus klase, atlikti talpos ir greicio testai, sukurtas setup failas. @section autotoc_md19 v3.0 versijos testavimas @subsection autotoc_md20 Laiko ir talpos testavimai @subsection autotoc_md21 Pastaba: laikas matuojamas sekundemis.

|--|

std::vector sparta

Mano vector sparta

std::vector perskirstymai

Mano vector perskirstymai

10000

0.000414

0.000483

14

14

100000

0.002356

0.002296

17

17

1000000

0.026129

0.021990

20

20

10000000

0.249020

0.236822

24

24

100000000

2.473831

2.340248

27

27

1.12.1 `std::vector` ir mano vektoriaus klases vidutinio programos veikimo laiko palyginimas

1.12.1.1 Pastaba: Kiekvienas greitis - vidutinis 3 testu laikas (sek.).

Studentu kiekis	<code>std::vector</code>	Mano <code>vvector</code>
100000	0.588639	1.102664
1000000	5.930762	7.65413
10000000	90.06457	99.740036

1.13 Rule of Five realizacija

Metodas	Paskirtis
<code>Studentas (const Studentas&)</code>	Kopijavimo konstruktorius
<code>Studentas& operator=(const Studentas&)</code>	Kopijavimo priskyrimo operatorius
<code>Studentas (Studentas&&)</code>	Perkelimo konstruktorius
<code>Studentas& operator=(Studentas&&)</code>	Perkelimo priskyrimo operatorius
<code>~Studentas ()</code>	Destruktorius

1.14 IO operatoriai

- `operator>>` leidžia nuskaityti studenta is srauto.
- `operator<<` leidžia isvesti studenta i srauta.

1.15 Rule of Five ir IO operatorius testai

Visi „Rule of Five“ metodai bei IO operatoriai testuojami naudojant `assert`:

- Default konstruktorius
- Kopijavimo konstruktorius ir operatorius
- Perkėlimo konstruktorius ir operatorius
- IO operatoriai (`>>` ir `<<`)

1.16 V1.1 versijos testavimai ir palyginimas su V1.0 versija (Nauja)

1.16.0.1 Pastaba: Kiekvienas greitis - vidutinis 3 testu laikas (sek.). Programoje naudojamas konteineris - vektorius. Paleidžiama 3 strategija.

Kompiliavimo veliava	Greitis 100000 (Naudojant struct)	Greitis 1000000 (Naudojant struct)	Exe failo dydis (Naudojant struct)	Greitis 100000 (Naudojant class)	Greitis 1000000 (Naudojant class)	Exe failo dydis (Naudojant class)
Jokia veliava	0.444916	5.298091	544 KB	0.728926	15.676393	544 KB
O1	0.448639	5.430932	544 KB	0.743053	15.877131	532 KB
O2	0.454392	5.462449	544 KB	0.729011	15.71323	532 KB
O3	0.451663	5.422904	538 KB	0.734674	15.707292	532 KB

1.17 V1.0 versijos testavimai (Sena)

1.18 Programos testavimo rezultatai pradiniam v1.0 release

1.18.1 1000 Studentų analizė

Viso faile yra 18 namų darbų pažymių

1.18.1.1 Nuskaitymas (sek.):

	Vector	List	Deque
1 testas	0.0072475	0.0082808	0.0145365
2 testas	0.0044263	0.0043628	0.007807
3 testas	0.0046425	0.0046128	0.0067956

1.18.1.2 Rusiavimas faile (sek.):

	Vector	List	Deque
1 testas	0.004638	0.0032458	0.0077588
2 testas	0.0045471	0.0035307	0.0096666
3 testas	0.0045742	0.0031883	0.0076848

1.18.1.3 Rusiavimas i kietekus ir vargsiukus (sek.):

	Vector	List	Deque
1 testas	0.0006764	0.0004916	0.0007002
2 testas	0.0006803	0.00069	0.0008425
3 testas	0.0006776	0.0004679	0.0008806

1.18.1.4 Vidurkiai (sek.):

	Vector	List	Deque
Bendras failo nuskaitymo laikas	0.016316	0.017256	0.029139
Vidutinis failo nuskaitymo laikas	0.005439	0.005752	0.009713
Bendras failo rusiavimo laikas	0.013759	0.009965	0.025110

Vidutinis failo rusiavimo laikas	0.002293	0.001661	0.004185
Bendras failo rusiavimo i kietekus ir vargsiukus laikas	0.002034	0.001650	0.002423
Vidutinis failo rusiavimo i kietekus ir vargsiukus laikas	0.000678	0.000550	0.000808
Bendras programos veikimo laikas	0.032109	0.028871	0,282662
Vidutinis programos veikimo laikas	0.010703	0.0096236667	0.09422066

1.18.2 10000 Studentu analize

Viso faile yra 18 namų darbų pažymių

1.18.2.1 Nuskaitymas (sek.):

	Vector	List	Deque
1 testas	0.0452272	0.0787392	0.0722085
2 testas	0.0491618	0.0706549	0.0647305
3 testas	0.0396006	0.0719151	0.0690108

1.18.2.2 Rusiavimas faile (sek.):

	Vector	List	Deque
1 testas	0.0064087	0.0836519	0.102892
2 testas	0.0054171	0.083851	0.10707
3 testas	0.006708	0.0902805	0.0987082

1.18.2.3 Rusiavimas i kietekus ir vargsiukus (sek.):

	Vector	List	Deque
1 testas	0.0550808	0.0103057	0.0074288
2 testas	0.0555215	0.0094287	0.0124954
3 testas	0.0573123	0.0122553	0.007408

1.18.2.4 Vidurkliai (sek.):

	Vector	List	Deque
Bendras failo nuskaitymo laikas	0.133990	0.221309	0.205950
Vidutinis failo nuskaitymo laikas	0.044663	0.073770	0.068650
Bendras failo rusiavimo laikas	0.167915	0.257783	0.308670
Vidutinis failo rusiavimo laikas	0.027986	0.042964	0.051445
Bendras failo rusiavimo i kietekus ir vargsiukus laikas	0.018534	0.031990	0.027332
Vidutinis failo rusiavimo i kietekus ir vargsiukus laikas	0.006178	0.010663	0.009111
Bendras programos veikimo laikas	0.320439	0.511082	0.541952
Vidutinis programos veikimo laikas	0.106813	0.1703606667	0.18065066

1.18.3 100000 Studentu analize

Viso faile yra 14 namų darbų pažymių

1.18.3.1 Nuskaitymas (sek.):

	Vector	List	Deque
1 testas	0.333077	0.577193	0.530389
2 testas	0.321548	0.613278	0.533229
3 testas	0.325427	0.586978	0.537504

1.18.3.2 Rusiavimas faile (sek.):

	Vector	List	Deque
1 testas	0.673029	1.0759	1.31066
2 testas	0.680436	1.07855	1.26595
3 testas	0.678414	1.08067	1.28526

1.18.3.3 Rusiavimas i kietekus ir vargsiukus (sek.):

	Vector	List	Deque
1 testas	0.0569962	0.0931008	0.0756288
2 testas	0.0575559	0.10202	0.0733271
3 testas	0.0557648	0.0904348	0.0774862

1.18.3.4 Vidurkiai (sek.):

	Vector	List	Deque
Bendras failo nuskaitymo laikas	0.980052	1.777449	1.601122
Vidutinis failo nuskaitymo laikas	0.326684	0.592483	0.533707
Bendras failo rusiavimo laikas	2.031878	3.235127	3.861866
Vidutinis failo rusiavimo laikas	0.338646	0.539188	0.643644
Bendras failo rusiavimo i kietekus ir vargsiukus laikas	0.170317	0.285555	0.226442
Vidutinis failo rusiavimo i kietekus ir vargsiukus laikas	0.056772	0.095185	0.075481
Bendras programos veikimo laikas	3.182247	5.298131	5.68943
Vidutinis programos veikimo laikas	1.060749	1.7660436	1.8967466

1.18.4 1000000 Studentu analize

Viso faile yra 19 namų darbų pažymių

1.18.4.1 Nuskaitymas (sek.):

	Vector	List	Deque
1 testas	3.95492	7.15517	6.40691
2 testas	3.87019	6.7632	6.42821
3 testas	3.84463	6.80788	6.49617

1.18.4.2 Rusiavimas faile (sek.):

	Vector	List	Deque
1 testas	9.88418	13.6062	14.4732
2 testas	9.8109	12.9613	14.8146
3 testas	9.82957	12.8813	14.3128

1.18.4.3 Rusiavimas i kietekus ir vargsiukus (sek.):

	Vector	List	Deque
1 testas	0.574952	0.919172	0.90257
2 testas	0.574212	0.953543	0.921527
3 testas	0.574577	1.02545	0.940349

1.18.4.4 Vidurkiai (sek.):

	Vector	List	Deque
Bendras failo nuskaitymo laikas	11.669734	20.726261	19.331293
Vidutinis failo nuskaitymo laikas	3.889911	6.908754	6.443764
Bendras failo rusiavimo laikas	29.524652	39.448779	43.600653
Vidutinis failo rusiavimo laikas	4.920775	6.574797	7.266776
Bendras failo rusiavimo i kietekus ir vargsiukus laikas	1.723741	2.898165	2.764446
Vidutinis failo rusiavimo i kietekus ir vargsiukus laikas	0.574580	0.966055	0.921482
Bendras programos veikimo laikas	42.918127	63.073205	65.696392
Vidutinis programos veikimo laikas	14.3060423333	21.02440166	21.89879733

1.18.5 10000000 Studentu analize

Viso faile yra 19 namų darbų pažymių

1.18.5.1 Nuskaitymas (sek.):

	Vector	List	Deque
1 testas	44.5152	55.7881	45.5688
2 testas	43.9039	45.815	45.0055
3 testas	41.9285	46.5904	50.5224

1.18.5.2 Rusiavimas faile (sek.):

	Vector	List	Deque
1 testas	119.483	115.353	130.304
2 testas	119.413	115.038	128.954
3 testas	119.039	115.01	129.462

1.18.5.3 Rusiavimas i kietekus ir vargsiukus (sek.):

	Vector	List	Deque
1 testas	6.28305	11.1458	8.20431
2 testas	6.71133	10.1772	8.78032
3 testas	6.47369	9.98839	8.81014

1.18.5.4 Vidurkiai (sek.):

	Vector	List	Deque
Bendras failo nuskaitymo laikas	130.347577	148.193507	141.096647
Vidutinis failo nuskaitymo laikas	43.449192	49.397836	47.032216
Bendras failo rusiavimo laikas	357.935381	345.401288	388.718866
Vidutinis failo rusiavimo laikas	59.655897	57.566881	64.786478
Bendras failo rusiavimo i kietekus ir vargsiukus laikas	19.468074	31.311419	25.794769
Vidutinis failo rusiavimo i kietekus ir vargsiukus laikas	6.489358	10.437140	8.598256
Bendras programos veikimo laikas	507.751032	524.906214	555.610282
Vidutinis programos veikimo laikas	169.250344	174.968738	185.20342733

1.19 Programos rusiavimo i kietekus ir vargsiukus greitis priklausomai nuo strategiju rezultatai

1.19.1 1 strategija

1.19.1.1 1000 Studentu analize

Viso faile yra 18 namų darbų pažymių

	Vector	List	Deque
1 testas	0.002109	0.0012078	0.0014959
2 testas	0.0029129	0.0018549	0.0015636
3 testas	0.002204	0.0017481	0.0021251
Bendras laikas	0.007226	0.004811	0.005185
Vidutinis laikas	0.002409	0.001604	0.001728

1.19.1.2 Nuotraukos

1.19.1.2.1 Vector

1.19.1.2.2 List

1.19.1.2.3 Deque

1.19.1.3 10000 Studentu analize

Viso faile yra 18 namų darbų pažymių

	Vector	List	Deque
1 testas	0.0160859	0.0126965	0.0116877
2 testas	0.0141176	0.0134405	0.0112872
3 testas	0.0168775	0.0160783	0.0112756
Bendras laikas	0.047081	0.042215	0.034251
Vidutinis laikas	0.015694	0.014072	0.011417

1.19.1.4 Nuotraukos

1.19.1.4.1 Vector

1.19.1.4.2 List

1.19.1.4.3 Deque

1.19.1.5 100000 Studentu analize

Viso faile yra 14 namų darbų pažymių

	Vector	List	Deque
1 testas	0.134149	0.136899	0.113251
2 testas	0.133631	0.142888	0.112049
3 testas	0.129876	0.131859	0.121555
Bendras laikas	0.397656	0.411647	0.346855
Vidutinis laikas	0.132552	0.137216	0.115618

1.19.1.6 Nuotraukos

1.19.1.6.1 Vector

1.19.1.6.2 List

1.19.1.6.3 Deque

1.19.1.7 1000000 Studentu analize

Viso faile yra 19 namų darbų pažymių

	Vector	List	Deque
1 testas	1.5492	1.55088	1.45731
2 testas	1.51583	1.61241	1.30246
3 testas	1.49225	1.49797	1.39939
Bendras laikas	4.557273	4.661263	4.159162
Vidutinis laikas	1.519091	1.553754	1.386387

1.19.1.8 Nuotraukos**1.19.1.8.1 Vector****1.19.1.8.2 List****1.19.1.8.3 Deque****1.19.1.9 10000000 Studentu analize**

Viso faile yra 19 namų darbų pažymių

	Vector	List	Deque
1 testas	15.166	12.4339	12.5972
2 testas	14.1786	13.1281	11.9944
3 testas	13.9391	13.1281	11.968
Bendras laikas	43.283757	39.586256	36.559604
Vidutinis laikas	14.427919	13.195419	12.186535

1.19.1.10 Nuotraukos

1.19.1.10.1 Vector

1.19.1.10.2 List

1.19.1.10.3 Deque

1.19.2 2 strategija

1.19.2.1 1000 Studentu analize

Viso faile yra 18 namų darbų pažymių

	Vector	List	Deque
1 testas	0.0020277	0.0016194	0.0018491
2 testas	0.0019987	0.0010802	0.0020192
3 testas	0.0020458	0.0011127	0.0019972
Bendras laikas	0.006072	0.003812	0.005866
Vidutinis laikas	0.002024	0.001271	0.001955

1.19.2.2 Nuotraukos

1.19.2.2.1 Vector

1.19.2.2.2 List

1.19.2.2.3 Deque

1.19.2.3 10000 Studentu analize

Viso faile yra 18 namų darbų pažymių

	Vector	List	Deque
1 testas	0.0836527	0.0112158	0.0241349
2 testas	0.0217737	0.0136404	0.023094
3 testas	0.0217998	0.0179673	0.0218672
Bendras laikas	0.127226	0.042824	0.069096
Vidutinis laikas	0.042409	0.014275	0.023032

1.19.2.4 Nuotraukos**1.19.2.4.1 Vector****1.19.2.4.2 List****1.19.2.4.3 Deque****1.19.2.5 100000 Studentu analize**

Viso faile yra 14 namų darbų pažymių

	Vector	List	Deque
1 testas	0.30975	0.177184	0.280622
2 testas	0.260698	0.188116	0.28941
3 testas	0.251467	0.181947	0.294386
Bendras laikas	0.821915	0.547246	0.864418
Vidutinis laikas	0.273972	0.182415	0.288139

1.19.2.6 Nuotraukos**1.19.2.6.1 Vector****1.19.2.6.2 List****1.19.2.6.3 Deque****1.19.2.7 1000000 Studentu analize**

Viso faile yra 19 namų darbų pažymių

	Vector	List	Deque
1 testas	3.25533	2.65689	3.86107
2 testas	3.19483	2.59375	3.61851
3 testas	3.22155	2.47487	3.83965
Bendras laikas	9.671704	7.725517	11.319226
Vidutinis laikas	3.223901	2.575172	3.773075

1.19.2.8 Nuotraukos**1.19.2.8.1 Vector****1.19.2.8.2 List****1.19.2.8.3 Deque****1.19.2.9 10000000 Studentu analize**

Viso faile yra 19 namų darbų pažymių

	Vector	List	Deque
1 testas	25.9085	26.6098	29.7575
2 testas	26.4085	22.6966	29.5761
3 testas	29.7513	23.138	30.6643
Bendras laikas	82.068295	72.444385	89.997919
Vidutinis laikas	27.356098	24.148128	29.999306

1.19.2.10 Nuotraukos

1.19.2.10.1 Vector

1.19.2.10.2 List

1.19.2.10.3 Deque

1.19.3 3 strategija

Pagal 1 strategija naudojant std::stable_partition

1.19.3.1 1000 Studentu analize

Viso faile yra 18 namų darbų pažymių

	Vector	List	Deque
1 testas	0.001565	0.0017377	0.0019046
2 testas	0.0017442	0.0027647	0.0019809
3 testas	0.0017182	0.0017408	0.0021929
Bendras laikas	0.005027	0.006243	0.006078
Vidutinis laikas	0.001676	0.002081	0.002026

1.19.3.2 Nuotraukos

1.19.3.2.1 Vector

1.19.3.2.2 List

1.19.3.2.3 Deque

1.19.3.3 10000 Studentu analize

Viso faile yra 18 namų darbų pažymių

	Vector	List	Deque
1 testas	0.0090895	0.0222504	0.0195486
2 testas	0.0147448	0.021207	0.0163617
3 testas	0.0151281	0.0176257	0.0240077
Bendras laikas	0.038962	0.4521997	0.059918
Vidutinis laikas	0.012987	0.1507332	0.019973

1.19.3.4 Nuotraukos**1.19.3.4.1 Vector****1.19.3.4.2 List****1.19.3.4.3 Deque****1.19.3.5 100000 Studentu analize**

Viso faile yra 14 namų darbų pažymių

	Vector	List	Deque
1 testas	0.101376	0.10475	0.122028
2 testas	0.0868336	0.103708	0.122189
3 testas	0.0868577	0.10367	0.126421
Bendras laikas	0.275067	0.312128	0.370638
Vidutinis laikas	0.091689	0.104043	0.123546

1.19.3.6 Nuotraukos**1.19.3.6.1 Vector****1.19.3.6.2 List****1.19.3.6.3 Deque****1.19.3.7 1000000 Studentu analize**

Viso faile yra 19 namų darbų pažymių

	Vector	List	Deque
1 testas	0.962317	1.11344	1.32951
2 testas	0.967044	1.11441	1.31631
3 testas	0.971096	1.12013	1.29266
Bendras laikas	2.900457	3.347979	3.938470
Vidutinis laikas	0.966819	1.115993	1.312823

1.19.3.8 Nuotraukos**1.19.3.8.1 Vector****1.19.3.8.2 List****1.19.3.8.3 Deque****1.19.3.9 10000000 Studentu analize**

Viso faile yra 19 namų darbų pažymių

	Vector	List	Deque
1 testas	11.3622	12.7829	15.1868
2 testas	11.2361	16.5202	16.1749
3 testas	11.024	13.9905	17.4642
Bendras laikas	33.622268	43.293562	48.825895
Vidutinis laikas	11.207423	14.431187	16.275298

1.19.3.10 Nuotraukos

1.19.3.10.1 Vector

1.19.3.10.2 List

1.19.3.10.3 Deque

1.20 5 failų kūrimas

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

vector< T >	27
Zmogus	28
Studentas	25

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Studentas	25
vector< T >	27
Zmogus	28

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

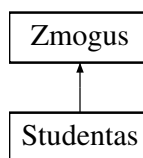
santrauka.h	29
Vector/ funkcijos.h	29
Vector/ studentas.h	30
Vector/ vector.h	32
Vector/ zmogus.h	35

Chapter 5

Class Documentation

5.1 Studentas Class Reference

Inheritance diagram for Studentas:



Public Member Functions

- **Studentas** (std::string vardas, std::string pavarde, [vector](#)< int > pazymiai, int egzamino_pazymys)
- void **clear** ()
- **Studentas** (const [Studentas](#) &studentas)
- [Studentas](#) & **operator=** (const [Studentas](#) &studentas)
- **Studentas** ([Studentas](#) &&studentas) noexcept
- [Studentas](#) & **operator=** ([Studentas](#) &&studentas) noexcept
- void [setVardas](#) (const std::string &vardas) override
- std::string [getVardas](#) () const override
- void [setPavarde](#) (const std::string &pavarde) override
- std::string [getPavarde](#) () const override
- [vector](#)< int > [getPazymiai](#) () const
- int [getEgzaminoPazymys](#) () const
- double [getGalutinisPazymysVid](#) () const
- double [getGalutinisPazymysMed](#) () const
- void [setPazymiai](#) ([vector](#)< int > pazymiai)
- void [setEgzaminoPazymys](#) (int egzamino_pazymys)
- void [addPazymys](#) (int pazymys)
- void [removeLastPazymys](#) (int pazymys)
- void [galutinis_pazymys_vid](#) ()
- void [galutinis_pazymys_med](#) ()
- bool [vardoPaieska](#) () const
- bool [pavardesPaieska](#) () const
- bool **operator==** (const [Studentas](#) &s) const
- bool **operator!=** (const [Studentas](#) &s) const

Public Member Functions inherited from [Zmogus](#)

- **Zmogus** (const std::string &vardas, const std::string &pavarde)

Friends

- std::istream & **operator**>> (std::istream &is, [Studentas](#) &studentas)
- std::ostream & **operator**<< (std::ostream &os, const [Studentas](#) &studentas)

Additional Inherited Members

Protected Attributes inherited from [Zmogus](#)

- std::string vardas_
- std::string pavarde_

5.1.1 Member Function Documentation

5.1.1.1 getPavarde()

```
std::string Studentas::getPavarde () const [inline], [override], [virtual]
```

Implements [Zmogus](#).

5.1.1.2 getVardas()

```
std::string Studentas::getVardas () const [inline], [override], [virtual]
```

Implements [Zmogus](#).

5.1.1.3 setPavarde()

```
void Studentas::setPavarde (  
    const std::string & pavarde) [inline], [override], [virtual]
```

Implements [Zmogus](#).

5.1.1.4 setVardas()

```
void Studentas::setVardas (  
    const std::string & vardas) [inline], [override], [virtual]
```

Implements [Zmogus](#).

The documentation for this class was generated from the following files:

- Vector/studentas.h
- Vector/studentas.cpp

5.2 vector< T > Class Template Reference

Public Types

- typedef size_t **size_type**
- typedef T **value_type**
- typedef T & **reference**
- typedef const T & **const_reference**
- typedef T * **iterator**
- typedef const T * **const_iterator**
- typedef std::reverse_iterator< iterator > **reverse_iterator**
- typedef std::reverse_iterator< const_iterator > **const_reverse_iterator**

Public Member Functions

- **vector** (size_type n, const T &t=T{})
- **vector** (const [vector](#) &v)
- template<class InputIterator>
 vector (InputIterator first, InputIterator last)
- **vector** ([vector](#) &&v) noexcept
- **vector** (std::initializer_list< T > il)
- [vector](#) & **operator=** (const [vector](#) &other)
- [vector](#) & **operator=** ([vector](#) &&other) noexcept
- iterator **begin** ()
- const_iterator **begin** () const
- iterator **end** ()
- const_iterator **end** () const
- const_iterator **cbegin** () const
- const_iterator **cend** () const
- reverse_iterator **rbegin** ()
- reverse_iterator **rend** ()
- const_reverse_iterator **crbegin** () const
- const_reverse_iterator **crend** () const
- size_type **size** () const
- size_type **max_size** () const
- void **resize** (size_type sz)
- void **resize** (size_type sz, const value_type &value)
- size_type **capacity** () const
- bool **empty** () const noexcept
- void **reserve** (size_type n)
- void **shrink_to_fit** ()
- T & **operator[]** (size_type n)
- const T & **operator[]** (size_type n) const
- reference **at** (size_type n)
- const_reference **at** (size_type n) const
- reference **front** ()
- const_reference **front** () const
- reference **back** ()
- const_reference **back** () const
- value_type * **data** () noexcept
- const value_type * **data** () const noexcept
- template<class InputIterator>
 void **assign** (InputIterator first, InputIterator last)

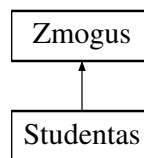
- void **assign** (size_type n, const value_type &val)
- void **assign** (std::initializer_list< value_type > il)
- void **push_back** (const value_type &t)
- void **push_back** (value_type &&val)
- void **pop_back** ()
- iterator **insert** (iterator pos, const T &value)
- iterator **insert** (iterator pos, size_type count, const T &value)
- iterator **erase** (iterator position)
- iterator **erase** (iterator first, iterator last)
- void **swap** (vector &x)
- void **clear** () noexcept
- bool **operator==** (const vector< T > &other) const
- bool **operator!=** (const vector< T > &other) const
- bool **operator<** (const vector< T > &other) const
- bool **operator<=** (const vector< T > &other) const
- bool **operator>** (const vector< T > &other) const
- bool **operator>=** (const vector< T > &other) const

The documentation for this class was generated from the following file:

- Vector/vector.h

5.3 Zmogus Class Reference

Inheritance diagram for Zmogus:



Public Member Functions

- **Zmogus** (const std::string &vardas, const std::string &pavarde)
- virtual void **setVardas** (const std::string &vardas)=0
- virtual std::string **getVardas** () const =0
- virtual void **setPavarde** (const std::string &pavarde)=0
- virtual std::string **getPavarde** () const =0

Protected Attributes

- std::string **vardas_**
- std::string **pavarde_**

The documentation for this class was generated from the following file:

- Vector/zmogus.h

Chapter 6

File Documentation

6.1 santrauka.h

```
00001 #include <vector>
00002 #include <iostream>
00003 #include <string>
00004 #include <fstream>
00005 #include <algorithm>
00006 #include <iomanip>
00007 #include <sstream>
00008 #include <limits>
00009 #include <random>
00010 #include <ctime>
00011 #include <chrono>
00012 #include <ostream>
00013 #include <chrono>
00014 #include <cstdlib>
00015 #include <stdexcept>
00016 #include <exception>
00017 #include <numeric>
00018 #include <iterator>
00019 #include <ios>
00020 #include <filesystem>
00021 #include <list>
00022 #include <deque>
00023 #include <cassert>
00024
00025 using std::cerr;
00026 using std::cin;
00027 using std::cout;
00028 using std::deque;
00029 using std::endl;
00030 using std::fixed;
00031 using std::ifstream;
00032 using std::stringstream;
00033 using std::left;
00034 using std::list;
00035 using std::max;
00036 using std::move;
00037 using std::ofstream;
00038 using std::ostream;
00039 using std::right;
00040 using std::setprecision;
00041 using std::setw;
00042 using std::sort;
00043 using std::string;
00044 using std::stringstream;
00045 using std::to_string;
00046 // using std::vector;
00047 using std::chrono::duration;
00048 using std::chrono::high_resolution_clock;
```

6.2 funkcijos.h

```
00001 #ifndef FUNCTIONS_H
00002 #define FUNCTIONS_H
```

```

00003 #include "../santrauka.h"
00004 #include "studentas.h"
00005 #include "vector.h"
00006
00007 const string aplankalo_pavadinimas = "NuskaitymoFailai";
00008 extern vector<std::chrono::duration<double>> testuLaikai;
00009 extern vector<std::chrono::duration<double>> kurimoLaikai;
00010 extern vector<std::chrono::duration<double>> rusiavimoLaikai;
00011 extern vector<std::chrono::duration<double>> kietekuLaikai;
00012 extern vector<std::chrono::duration<double>> vargsiukuLaikai;
00013 extern vector<std::chrono::duration<double>> vienoIsSesiuRusiavimoLaikai;
00014 extern int testuSkaicius;
00015 extern int kurimuSkaicius;
00016 extern int rusiavimoSkaicius;
00017 extern int vienoIsSesiuRusiavimoSkaicius;
00018 extern int kiekYraStudentuFaile;
00019 void aplankalas();
00020 int failo_generavimo_pasirinkimas(vector<string> &failai);
00021 void failo_generavimas(const string &failo_pavadinimas, long long studentu_skaicius);
00022 void parodytiEsamusFailus();
00023 string koki_faila_nuskaityti();
00024 int Meniu();
00025 void failo_nuskaitymas(vector<Studentas> &studentai, const string &failo_pavadinimas);
00026 void pabaiga(int pasirinkimas, int generuoti, int pasirinkimas_rusiavimui, int kaip_surusuoti);
00027 void nuskaitytiIsFailo(vector<Studentas> &studentai, string failo_pasirinkimas);
00028 bool ivestiStudenta(Studentas &laikinas);
00029 void generuotiVardaPavarde(Studentas &laikinas);
00030 void ivestiPazymius(Studentas &laikinas);
00031 void generuotiPazymius(Studentas &laikinas);
00032 void informacijos_ivedimas(vector<Studentas> &studentai, int pasirinkimas, string failo_pavadinimas);
00033 int koks_galutinis();
00034 int failas_ar_konsole();
00035 int rusiavimas(int skaiciavimo_budas);
00036 bool vardoRusiavimas(const Studentas &studentas, const Studentas &studentas2);
00037 bool pavardeRusiavimas(const Studentas &studentas, const Studentas &studentas2);
00038 bool vidurkioNuoMazRusiavimas(const Studentas &studentas, const Studentas &studentas2);
00039 bool vidurkioNuoDidRusiavimas(const Studentas &studentas, const Studentas &studentas2);
00040 bool medianosNuoMazRusiavimas(const Studentas &studentas, const Studentas &studentas2);
00041 bool medianosNuoDidRusiavimas(const Studentas &studentas, const Studentas &studentas2);
00042 int pagal_ka_rusiuoti_studentus_i_failus(vector<Studentas> &studentai, int skaiciavimo_budas);
00043 int papildomas_rusiavimas();
00044 void studentu_rusiavimas_3strategija(vector<Studentas> &studentai, int skaiciavimo_budas, int
    kaip_surusuoti, int rusiavimo_budas);
00045 void output(ostream &out, vector<Studentas> studentai, int skaiciavimo_budas, int isvedimo_budas);
00046 void rusiavimoIf(vector<Studentas> &studentai, int kaip_surusuoti, int skaiciavimo_budas);
00047 void terminalas(vector<Studentas> &studentai, int kaip_surusuoti, int skaiciavimo_budas);
00048 void failas(vector<Studentas> &studentai, int kaip_surusuoti, int skaiciavimo_budas);
00049 void spartosTestas();
00050
00051 #endif

```

6.3 studentas.h

```

00001 #ifndef STUDENTAS_H
00002 #define STUDENTAS_H
00003 #include "../santrauka.h"
00004 #include "zmogus.h"
00005 #include "vector.h"
00006
00007 class Studentas : public Zmogus
00008 {
00009 private:
00010     vector<int> pazymiai_;
00011     int egzamino_pazymys_ = 0;
00012     double galutinis_pazymys_vid_ = 0.0;
00013     double galutinis_pazymys_med_ = 0.0;
00014
00015 public:
00016     // Default konstruktorius
00017     Studentas()
00018         : Zmogus("Vardenis", "Pavardenis"),
00019           pazymiai_ {},
00020           egzamino_pazymys_{0},
00021           galutinis_pazymys_vid_{0.0},
00022           galutinis_pazymys_med_{0.0} {}
00023
00024     Studentas(std::string vardas, std::string pavarde, vector<int> pazymiai, int egzamino_pazymys)
00025         : Zmogus(vardas, pavarde),
00026           pazymiai_{pazymiai},
00027           egzamino_pazymys_{egzamino_pazymys} {}
00028
00029     void clear()
00030     {

```

```

00031     pazymiai_.clear();
00032     egzamino_pazymys_ = 0;
00033 }
00034 // Destruktorius
00035 ~Studentas() { clear(); }
00036
00037 // Copy konstruktorius
00038 Studentas(const Studentas &studentas)
00039 : Zmogus(studentas.vardas_, studentas.pavarde_),
00040   pazymiai_(studentas.pazymiai_),
00041   egzamino_pazymys_(studentas.egzamino_pazymys_),
00042   galutinis_pazymys_vid_(studentas.galutinis_pazymys_vid_),
00043   galutinis_pazymys_med_(studentas.galutinis_pazymys_med_) {}
00044
00045 // Copy assignment operatorius
00046 Studentas &operator=(const Studentas &studentas)
00047 {
00048     if (this != &studentas)
00049     {
00050         vardas_ = studentas.vardas_;
00051         pavarde_ = studentas.pavarde_;
00052         pazymiai_ = studentas.pazymiai_;
00053         egzamino_pazymys_ = studentas.egzamino_pazymys_;
00054         galutinis_pazymys_vid_ = studentas.galutinis_pazymys_vid_;
00055         galutinis_pazymys_med_ = studentas.galutinis_pazymys_med_;
00056     }
00057     return *this;
00058 }
00059
00060 // Move konstruktorius
00061 Studentas(Studentas &&studentas) noexcept
00062 : Zmogus(std::move(studentas.vardas_), std::move(studentas.pavarde_)),
00063   pazymiai_(std::move(studentas.pazymiai_)),
00064   egzamino_pazymys_(std::move(studentas.egzamino_pazymys_)),
00065   galutinis_pazymys_vid_(std::move(studentas.galutinis_pazymys_vid_)),
00066   galutinis_pazymys_med_(std::move(studentas.galutinis_pazymys_med_)) {}
00067
00068 // Move assignment operatorius
00069 Studentas &operator=(Studentas &&studentas) noexcept
00070 {
00071     if (this != &studentas)
00072     {
00073         vardas_ = std::move(studentas.vardas_);
00074         pavarde_ = std::move(studentas.pavarde_);
00075         pazymiai_ = std::move(studentas.pazymiai_);
00076         egzamino_pazymys_ = std::move(studentas.egzamino_pazymys_);
00077         galutinis_pazymys_vid_ = std::move(studentas.galutinis_pazymys_vid_);
00078         galutinis_pazymys_med_ = std::move(studentas.galutinis_pazymys_med_);
00079     }
00080     return *this;
00081 }
00082
00083 // IO operatoriai
00084 friend std::istream &operator>>(std::istream &, Studentas &);
00085 friend std::ostream &operator<<(std::ostream &, const Studentas &);
00086
00087 void setVardas(const std::string &vardas) override { vardas_ = vardas; }
00088 std::string getVardas() const override { return vardas_; }
00089
00090 void setPavarde(const std::string &pavarde) override { pavarde_ = pavarde; }
00091 std::string getPavarde() const override { return pavarde_; }
00092
00093 vector<int> getPazymiai() const { return pazymiai_; }
00094 int getEgzaminoPazymys() const { return egzamino_pazymys_; }
00095 double getGalutinisPazymysVid() const { return galutinis_pazymys_vid_; }
00096 double getGalutinisPazymysMed() const { return galutinis_pazymys_med_; }
00097
00098 void setPazymiai(vector<int> pazymiai) { pazymiai_ = pazymiai; }
00099 void setEgzaminoPazymys(int egzamino_pazymys) { egzamino_pazymys_ = egzamino_pazymys; }
00100 void addPazymys(int pazymys) { pazymiai_.push_back(pazymys); }
00101 void removeLastPazymys(int pazymys) { pazymiai_.pop_back(); }
00102
00103 void galutinis_pazymys_vid();
00104 void galutinis_pazymys_med();
00105
00106 bool vardoPaieska() const { return vardas_.find("Vardas") == 0; }
00107 bool pavardesPaieska() const { return pavarde_.find("Pavarde") == 0; }
00108
00109 bool operator==(const Studentas &s) const
00110 {
00111     if (vardas_ == s.vardas_ && pavarde_ == s.pavarde_ && pazymiai_ == s.pazymiai_ &&
00112         egzamino_pazymys_ == s.egzamino_pazymys_ && galutinis_pazymys_vid_ ==
00113         s.galutinis_pazymys_vid_ && galutinis_pazymys_med_ == s.galutinis_pazymys_med_)
00114     {
00115         return true;
00116     }
00117     else

```

```

00117         return false;
00118     }
00119
00120     bool operator!=(const Studentas &s) const
00121     {
00122         if (vardas_ != s.vardas_ || pavarde_ != s.pavarde_ || pazymiai_ != s.pazymiai_ ||
00123             egzamino_pazymys_ != s.egzamino_pazymys_ || galutinis_pazymys_vid_ !=
00124             s.galutinis_pazymys_vid_ || galutinis_pazymys_med_ != s.galutinis_pazymys_med_)
00125         {
00126             return true;
00127         }
00128         else
00129             return false;
00130     };
00131
00132 #endif

```

6.4 vector.h

```

00001 #ifndef VECTOR_H
00002 #define VECTOR_H
00003
00004 #include <iostream>
00005 #include <memory>
00006 #include <algorithm>
00007 #include <limits>
00008 #include <initializer_list>
00009
00010 template <typename T>
00011 class vector
00012 {
00013 public:
00014     typedef size_t size_type;
00015     typedef T value_type;
00016     typedef T &reference;
00017     typedef const T &const_reference;
00018     typedef T *iterator;
00019     typedef const T *const_iterator;
00020     typedef std::reverse_iterator<iterator> reverse_iterator;
00021     typedef std::reverse_iterator<const_iterator> const_reverse_iterator;
00022
00023     vector() { create(); }
00024     explicit vector(size_type n, const T &t = T{}) { create(n, t); }
00025     vector(const vector &v) { create(v.begin(), v.end()); }
00026     template <class InputIterator>
00027     vector(InputIterator first, InputIterator last) { create(first, last); }
00028     vector(vector &&v) noexcept
00029     {
00030         dat = v.dat;
00031         avail = v.avail;
00032         limit = v.limit;
00033         v.dat = v.avail = v.limit = nullptr;
00034     }
00035     vector(std::initializer_list<T> il) { create(il.begin(), il.end()); }
00036     ~vector() { uncreate(); }
00037
00038     vector &operator=(const vector &other)
00039     {
00040         if (this != &other)
00041         {
00042             uncreate();
00043             create(other.begin(), other.end());
00044         }
00045         return *this;
00046     }
00047
00048     vector &operator=(vector &&other) noexcept
00049     {
00050         if (this != &other)
00051         {
00052             uncreate();
00053             std::swap(dat, other.dat);
00054             std::swap(avail, other.avail);
00055             std::swap(limit, other.limit);
00056         }
00057         return *this;
00058     }
00059
00060     iterator begin() { return dat; }
00061     const_iterator begin() const { return dat; }
00062     iterator end() { return avail; }
00063     const_iterator end() const { return avail; }

```

```

00064     const_iterator cbegin() const { return dat; }
00065     const_iterator cend() const { return avail; }
00066     reverse_iterator rbegin() { return reverse_iterator(avail); }
00067     reverse_iterator rend() { return reverse_iterator(dat); }
00068     const_reverse_iterator crbegin() const { return const_reverse_iterator(avail); }
00069     const_reverse_iterator crend() const { return const_reverse_iterator(dat); }
00070
00071     size_type size() const { return avail - dat; }
00072     size_type max_size() const { return std::numeric_limits<size_type>::max(); }
00073     void resize(size_type sz)
00074     {
00075         if (sz < size())
00076         {
00077             iterator it = dat + sz;
00078             while (it != avail)
00079                 alloc.destroy(it++);
00080             avail = dat + sz;
00081         }
00082         else if (sz > capacity())
00083         {
00084             grow(sz);
00085             std::uninitialized_fill(avail, dat + sz, value_type());
00086             avail = dat + sz;
00087         }
00088         else if (sz > size())
00089         {
00090             std::uninitialized_fill(avail, dat + sz, value_type());
00091             avail = dat + sz;
00092         }
00093     }
00094
00095     void resize(size_type sz, const value_type &value)
00096     {
00097         if (sz > capacity())
00098             grow(sz);
00099         if (sz > size())
00100         {
00101             insert(end(), sz - size(), value);
00102         }
00103         else if (sz < size())
00104         {
00105             avail = dat + sz;
00106         }
00107     }
00108
00109     size_type capacity() const { return limit - dat; }
00110     bool empty() const noexcept { return size() == 0; }
00111     void reserve(size_type n)
00112     {
00113         if (n > capacity())
00114             grow(n);
00115     }
00116     void shrink_to_fit()
00117     {
00118         if (limit > avail)
00119             limit = avail;
00120     }
00121
00122     T &operator[](size_type n) { return dat[n]; }
00123     const T &operator[](size_type n) const { return dat[n]; }
00124     reference at(size_type n)
00125     {
00126         if (n >= size())
00127             throw std::out_of_range("Index out of range");
00128         return dat[n];
00129     }
00130     const_reference at(size_type n) const
00131     {
00132         if (n >= size())
00133             throw std::out_of_range("Index out of range");
00134         return dat[n];
00135     }
00136     reference front() { return dat[0]; }
00137     const_reference front() const { return dat[0]; }
00138     reference back() { return dat[size() - 1]; }
00139     const_reference back() const { return dat[size() - 1]; }
00140     value_type *data() noexcept { return dat; }
00141     const value_type *data() const noexcept { return dat; }
00142
00143     template <class InputIterator>
00144     void assign(InputIterator first, InputIterator last)
00145     {
00146         uncreate();
00147         create(first, last);
00148     }
00149     void assign(size_type n, const value_type &val)
00150     {

```

```

00151         uncreate();
00152         create(n, val);
00153     }
00154     void assign(std::initializer_list<value_type> il)
00155     {
00156         uncreate();
00157         create(il.begin(), il.end());
00158     }
00159
00160     void push_back(const value_type &t)
00161     {
00162         if (avail == limit)
00163             grow();
00164         unchecked_append(t);
00165     }
00166     void push_back(value_type &&val)
00167     {
00168         if (avail == limit)
00169             grow();
00170         unchecked_append(std::move(val));
00171     }
00172
00173     void pop_back()
00174     {
00175         if (avail != dat)
00176             alloc.destroy(--avail);
00177     }
00178
00179     iterator insert(iterator pos, const T &value)
00180     {
00181         size_type index = pos - begin();
00182         size_type numNewElements = 1;
00183         if (size() + numNewElements > capacity())
00184             reserve((size() + numNewElements) * 2);
00185         std::move_backward(dat + index, avail, avail + numNewElements);
00186         dat[index] = value;
00187         avail += numNewElements;
00188         return dat + index;
00189     }
00190
00191     iterator insert(iterator pos, size_type count, const T &value)
00192     {
00193         size_type index = pos - begin();
00194         if (size() + count > capacity())
00195             reserve((size() + count) * 2);
00196
00197         pos = dat + index; // po galimos grow(), kad rodyklė būtų teisinga
00198         std::move_backward(pos, avail, avail + count);
00199         std::uninitialized_fill(pos, pos + count, value);
00200         avail += count;
00201         return pos;
00202     }
00203
00204     iterator erase(iterator position)
00205     {
00206         if (position < dat || position > avail)
00207             throw std::out_of_range("Index out of range");
00208         std::move(position + 1, avail, position);
00209         alloc.destroy(--avail);
00210         return position;
00211     }
00212
00213     iterator erase(iterator first, iterator last)
00214     {
00215         iterator new_available = std::uninitialized_copy(last, avail, first);
00216         iterator it = avail;
00217         while (it != new_available)
00218             alloc.destroy(--it);
00219         avail = new_available;
00220         return last;
00221     }
00222
00223     void swap(vector &x)
00224     {
00225         std::swap(dat, x.dat);
00226         std::swap(avail, x.avail);
00227         std::swap(limit, x.limit);
00228     }
00229
00230     void clear() noexcept { uncreate(); }
00231
00232     bool operator==(const vector<T> &other) const
00233     {
00234         return size() == other.size() && std::equal(begin(), end(), other.begin());
00235     }
00236     bool operator!=(const vector<T> &other) const { return !(*this == other); }
00237     bool operator<(const vector<T> &other) const

```



```

00238     {
00239         return std::lexicographical_compare(begin(), end(), other.begin(), other.end());
00240     }
00241     bool operator<=(const vector<T> &other) const { return !(other < *this); }
00242     bool operator>(const vector<T> &other) const
00243     {
00244         return std::lexicographical_compare(other.begin(), other.end(), begin(), end());
00245     }
00246     bool operator>=(const vector<T> &other) const { return !(*this < other); }
00247 private:
00248     iterator dat = nullptr;
00249     iterator avail = nullptr;
00250     iterator limit = nullptr;
00251     std::allocator<T> alloc;
00252
00253     void create() { dat = avail = limit = nullptr; }
00254     void create(size_type n, const T &val)
00255     {
00256         dat = alloc.allocate(n);
00257         avail = limit = dat + n;
00258         std::uninitialized_fill(dat, limit, val);
00259     }
00260     void create(const_iterator i, const_iterator j)
00261     {
00262         dat = alloc.allocate(j - i);
00263         limit = avail = std::uninitialized_copy(i, j, dat);
00264     }
00265     void uncreate()
00266     {
00267         if (dat)
00268         {
00269             iterator it = avail;
00270             while (it != dat)
00271                 alloc.destroy(--it);
00272             alloc.deallocate(dat, limit - dat);
00273         }
00274         dat = limit = avail = nullptr;
00275     }
00276     void grow(size_type new_capacity = 1)
00277     {
00278         size_type new_size = std::max(new_capacity, 2 * capacity());
00279         iterator new_data = alloc.allocate(new_size);
00280         iterator new_avail = std::uninitialized_copy(dat, avail, new_data);
00281         uncreate();
00282         dat = new_data;
00283         avail = new_avail;
00284         limit = dat + new_size;
00285     }
00286     void unchecked_append(const T &val)
00287     {
00288         alloc.construct(avail++, val);
00289     }
00290     void unchecked_append(T &&val)
00291     {
00292         alloc.construct(avail++, std::move(val));
00293     }
00294 };
00295 #endif

```

6.5 zmogus.h

```

00001 #ifndef ZMOGUS_H
00002 #define ZMOGUS_H
00003 #include "../santrauka.h"
00004 #include "vector.h"
00005
00006 class Zmogus
00007 {
00008 protected:
00009     std::string vardas_;
00010     std::string pavarde_;
00011
00012 public:
00013     Zmogus() = default;
00014     Zmogus(const std::string &vardas, const std::string &pavarde)
00015         : vardas_(vardas), pavarde_(pavarde) {}
00016
00017     virtual void setVardas(const std::string &vardas) = 0;
00018     virtual std::string getVardas() const = 0;
00019
00020     virtual void setPavarde(const std::string &pavarde) = 0;

```

```
00021     virtual std::string getPavarde() const = 0;
00022
00023     virtual ~Zmogus()
00024     {
00025         vardas_ = "";
00026         pavarde_ = "";
00027     }
00028 };
00029 #endif
```

Index

getPavarde
 Studentas, [26](#)
getVardas
 Studentas, [26](#)

setPavarde
 Studentas, [26](#)
setVardas
 Studentas, [26](#)
Studentas, [25](#)
 getPavarde, [26](#)
 getVardas, [26](#)
 setPavarde, [26](#)
 setVardas, [26](#)

Trecia versija, [1](#)

vector< T >, [27](#)
Vector/funkcijos.h, [29](#)
Vector/studentas.h, [30](#)
Vector/vector.h, [32](#)
Vector/zmogus.h, [35](#)

Zmogus, [28](#)